

Python for Data Science

Every data scientist you admire writes Python. Not because it's the only option - because it's the fastest path from raw idea to working analysis. This week you build that foundation, line by line.



Duration: 1 week



Hands-on: 4 coding labs in Google Colab



Assessment: Lab + Forum + Quiz

By end of this week, you will be able to

Write Python code using variables, data types, loops, conditionals, and functions

Use lists, tuples, and dictionaries to store and manipulate data

Import and apply NumPy, Pandas, and Matplotlib for data work

Load a CSV dataset, explore its structure, and handle missing values

Write reusable functions to clean and process data

Plot basic charts that communicate findings clearly

1. Why Python?

In 2011, Safaricom's data team ran most of their analysis in Excel. By 2018, they had moved to Python. The same shift happened at Equity Bank, at the Kenya National Bureau of Statistics, at KEMRI, at Apollo Agriculture. Python did not win because it is the most powerful language - it won because it sits at the intersection of *easy enough to learn fast* and *powerful enough to build anything*.

The libraries you will use this week - NumPy, Pandas, and Matplotlib - are the same ones running inside production systems at Google, Meta, and every African data science team worth knowing about. Learning them is not academic. It is vocational.

Before you write a single line: open colab.research.google.com, sign in with your Google account, and create a new notebook. Every lab this week runs there - nothing to install, works on any device including mobile.

2. Python Syntax & Variables

Python reads almost like English. There are no semicolons. Indentation defines structure. You can build something useful within your first hour.

int / float

Whole numbers and decimals. `age = 24` or `rate = 0.075`



str

Text in quotes. `county = "Nairobi"`



bool

True or False. Powers every conditional. `has_account = True`



None

Absence of a value. Represents missing data - you will see this constantly.

```
Lab 1 - Variables & types • Run in Colab
# Variables: store values with a name
county = "Nairobi"
population = 4397073
gdp_share = 0.45 # 45% of Kenya's GDP
has_4g = True # f-strings: the cleanest way to build messages
print(f"{county} has a population of {population:,}")
print(f"Contributes {gdp_share:.0%} of Kenya's GDP") # Check a variable's type
print(type(population)) # <class 'int'>
print(type(gdp_share)) # <class 'float'>
```

3. Lists, Tuples & Dictionaries

Real data is never a single value. It comes in collections. These three structures are how Python organises them - and you will use all three every single day as a data scientist.

```
Lab 2 - Data structures • Run in Colab
# — LISTS: ordered, changeable ————— counties = ["Nairobi", "Mombasa", "Kisumu", "Nakuru", "Eldoret"]
print(counties[0]) # "Nairobi" - indexing starts at 0
print(counties[-1]) # "Eldoret" - negative index = from end
print(counties[1:3]) # ["Mombasa", "Kisumu"] - slicing
counties.append("Thika") # add to end
print(len(counties)) # 6
# — TUPLES: ordered, fixed (immutable) ————— coordinates = (-1.286389, 36.817223)
# Nairobi lat/lon
lat, lon = coordinates # unpacking
print(f"Lat: {lat}, Lon: {lon}") # — DICTIONARIES: key-value pairs
learner = { "name": "Aisha", "county": "Mombasa", "score": 87, "has_laptop": True }
print(learner["name"]) # "Aisha"
learner["score"] = 92 # update a value
print(learner.keys()) # all keys
print(learner.values()) # all values
```

When to use which: Use a **list** when order matters and items can change (a column of survey responses). Use a **tuple** for fixed data that should not change (GPS coordinates,

RGB colour values). Use a **dictionary** when each value has a meaningful name (a single row of data with labelled fields).

4. Loops & Conditionals

Loops and conditionals are how you automate decisions at scale. Instead of manually flagging 10,000 M-Pesa transactions one by one, you write a loop that checks all of them in milliseconds.

```
Lab 3 - Loops & conditionals • Run in Colab
# — FOR LOOPS —————
m_pesa_transactions = [500, 12000, 250, 75000, 1800, 300000]
for amount in m_pesa_transactions:
    if amount > 50000:
        print(f"⚠ Flag for review: Ksh {amount:,}")
    elif amount > 10000:
        print(f"✓ Large: Ksh {amount:,}")
    else:
        print(f"Normal: Ksh {amount:,}")
# — WHILE LOOPS —————
attempts = 0
while attempts < 3:
    print(f"Login attempt {attempts + 1}")
    attempts += 1
# — LIST COMPREHENSIONS: compact loops —————
# Get only flagged transactions in one line
flagged = [t for t in m_pesa_transactions if t > 50000]
print(f"Flagged: {flagged}")
```

5. Functions & File Handling

A function is a reusable block of code you name and call whenever you need it. Every time you find yourself copy-pasting the same ten lines, that is a signal to write a function instead.

```
Lab 4 - Functions • Run in Colab
# — DEFINING FUNCTIONS ————— def
def flag_transaction(amount, threshold=50000):
    """Return True if a transaction exceeds the threshold."""
    return amount > threshold
# Call the function
print(flag_transaction(75000)) # True
print(flag_transaction(1500)) # False
print(flag_transaction(8000, threshold=5000)) # True (custom threshold)
# — A DATA CLEANING FUNCTION ————— def
def clean_county_name(name):
    """Standardise county names: strip spaces, title case."""
    if name is None:
        return "Unknown"
    return name.strip().title()
dirty_names = ["nairobi", "MOMBASA", None, "kisumu "]
clean_names = [clean_county_name(n) for n in dirty_names]
print(clean_names) # ['Nairobi', 'Mombasa', 'Unknown', 'Kisumu']
```

Good function habits: give it a verb name (`clean_name`, not `names`), write a one-line docstring, and make it do exactly one thing. Functions that do two things are bugs waiting to happen.

6. NumPy - Fast Maths on Arrays

NumPy (Numerical Python) lets you do maths on entire arrays of numbers at once - no loops needed. Under the hood, Pandas and Scikit-learn both run on NumPy. Understanding it makes everything else faster to learn.

```
Lab 5 - NumPy essentials • Run in Colab
import numpy as np # Simulate monthly mobile money transaction volumes
(millions, 2024) volumes = np.array([612, 589, 634, 601, 678, 655, 720, 698,
711, 743, 769, 801]) print(f"Annual total: {volumes.sum():,} million
transactions") print(f"Monthly mean: {volumes.mean():.1f}M") print(f"Std
deviation: {volumes.std():.1f}M") print(f"Peak month: {volumes.max()}M (index
{volumes.argmax()})") print(f"Slowest month: {volumes.min()}M") # Array
operations - no loop needed growth = np.diff(volumes) # month-on-month
change pct_growth = (growth / volumes[:-1]) * 100 # as percentage
print(f"\nAverage monthly growth: {pct_growth.mean():.2f}%") # Boolean
masking - months above average above_avg = volumes[volumes > volumes.mean()]
print(f"Above-average months: {len(above_avg)} of 12")
```

7. Pandas - The Workhorse of Data Science

If NumPy is the engine, Pandas is the dashboard. It gives you the DataFrame - a table of data with labelled rows and columns - and the tools to reshape, filter, summarise, and clean it. You will use Pandas in every remaining week of this programme.

```
Lab 6 - Pandas: loading & exploring data • Run in Colab
import pandas as pd # Load a CSV (use the Financial Inclusion dataset from
Week 1) df = pd.read_csv("train.csv") # — First look
print(df.shape) # (rows,
columns) df.head() # first 5 rows df.info() # column names, types, non-null
counts df.describe() # summary stats for numeric columns # — Selecting
columns
print(df["country"].value_counts()) # respondents per country
print(df["bank_account"].value_counts()) # our target variable # —
Filtering rows
kenya_df = df[df["country"] == "Kenya"] print(f"Kenya respondents: {len(kenya_df):,}") #
```

```

— Handling missing values —
print(df.isnull().sum()) # count NaNs per column df_clean =
df.dropna(subset=["bank_account"]) # drop rows missing target
df["age_of_respondent"] = df["age_of_respondent"].fillna(
df["age_of_respondent"].median() # fill age with median )
Lab 7 - Pandas: grouping & aggregating • Run in Colab
# Bank account ownership by country summary =
df.groupby("country")["bank_account"].value_counts( normalize=True
).unstack() * 100 print(summary.round(1)) # Average age of banked vs
unbanked respondents
print(df.groupby("bank_account")["age_of_respondent"].mean()) # Adding a new
calculated column df["is_young"] = df["age_of_respondent"] < 30
print(df["is_young"].value_counts())

```

8. Matplotlib - Telling Stories with Charts

A number means little until someone sees it. Matplotlib is Python's core visualisation library - from quick exploratory plots to publication-quality figures.

```

Lab 8 - Three essential chart types • Run in Colab
import matplotlib.pyplot as plt # — 1. BAR CHART: bank account ownership by
country ————— ownership = df.groupby("country")["bank_account"].apply(
lambda x: (x == "Yes").mean() * 100 ) fig, axes = plt.subplots(1, 3,
figsize=(15, 4)) ownership.plot(kind="bar", ax=axes[0], color="#1D9E75")
axes[0].set_title("Bank Account Ownership (%)") axes[0].set_ylabel("% with
bank account") axes[0].tick_params(axis="x", rotation=0) # — 2. HISTOGRAM:
age distribution —————
df["age_of_respondent"].hist(bins=20, ax=axes[1], color="#EF9F27")
axes[1].set_title("Age Distribution of Respondents")
axes[1].set_xlabel("Age") # — 3. PIE CHART: education level breakdown
————— df["education_level"].value_counts().plot( kind="pie",
ax=axes[2], autopct="%1.0f%%", startangle=90 ) axes[2].set_title("Education
Level") axes[2].set_ylabel("") plt.tight_layout() plt.show()

```

Chart choice guide: Use a **bar chart** to compare categories. Use a **histogram** to see the distribution of a numeric variable. Use a **line chart** for change over time. Use a **scatter plot** to explore the relationship between two numeric variables. Pie charts are useful for showing composition - but only with fewer than 5 categories.

Resources & References

Curated tools, documentation, tutorials, and reading for Week 2. Start with the official docs - they are better than most tutorials.

Official Documentation (Your First Stop)

[Freepython.org](https://python.org) ↗

The Python Tutorial - python.org

Official intro from the Python team. Covers everything in this week's lesson with authoritative detail. Chapters 3–5 are most relevant.

[Freepandas.pydata.org](https://pandas.pydata.org) ↗

10 Minutes to Pandas - Official Guide

The official 10-minute intro to Pandas. Covers DataFrames, selection, groupby, and missing data. Bookmark this - you will return to it often.

[Freenumpy.org](https://numpy.org) ↗

NumPy Quickstart - Official Guide

Arrays, indexing, slicing, broadcasting. The official quickstart is dense but complete - worth reading once carefully.

[Freematplotlib.org](https://matplotlib.org) ↗

Matplotlib Quick Start - Official Guide

Figures, axes, and plot types explained from the ground up. Includes interactive examples you can run directly in the browser.

Interactive Courses & Practice

[Freekaggle.com](https://freekaggle.com) ↗

Python - Kaggle Learn

Syntax, functions, lists, loops, and string manipulation in short browser-based exercises. Ideal as a parallel track to this week's labs.

[Freekaggle.com](https://freekaggle.com) ↗

Pandas - Kaggle Learn

Creating, reading, and writing DataFrames; indexing, selecting, and assigning data; grouping and sorting. Six short lessons with exercises.

[Freekaggle.com](https://freekaggle.com) ↗

Data Visualisation - Kaggle Learn

Seaborn-based (builds on Matplotlib). Line charts, bar charts, scatter plots, distributions, and choosing the right chart. Highly recommended.

[Free via Ngaodatacamp.com](https://free.via.Ngaodatacamp.com) ↗

Intro to Python for Data Science - DataCamp

Python basics + NumPy arrays. First course in DataCamp's Data Scientist track. Your Donates access covers this in full.

Cheat Sheets (Print & Keep)

[Free PDFpandas.pydata.org](https://free.PDFpandas.pydata.org) ↗

Official Pandas Cheat Sheet

Two-page reference covering all essential DataFrame operations. Print it. Pin it next to your laptop. You will use it weekly.

[Free PDF](#) matplotlib.org ↗

Matplotlib Cheat Sheets

Three sheets: beginner, intermediate, and tips & tricks. Official from the Matplotlib team. Invaluable when building visualisations from scratch.

[Free PDF](#) uw.edu ↗

NumPy Cheat Sheet - UW

Array creation, indexing, slicing, math operations, and boolean masking on a single page. Prepared by University of Washington.

Books & Deep Dives

[Free Online](#) github.io ↗

Python Data Science Handbook - NumPy Chapter

Jake VanderPlas. Deep dive into NumPy arrays, broadcasting, fancy indexing, and structured arrays. Free online.

[Free Online](#) github.io ↗

Python Data Science Handbook - Pandas Chapter

The most thorough free Pandas reference available. Covers data indexing, missing data, hierarchical indexing, and aggregation in depth.

[Book](#) wesmckinney.com ↗

Python for Data Analysis - Wes McKinney

Written by the creator of Pandas. The definitive reference. Third edition available free online. Chapters 4–9 cover everything in this week's lesson.

Academic References

- [1] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. - The original Pandas paper. scipy.org ↗
- [2] Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. - The NumPy paper with 10,000+ citations. nature.com ↗
- [3] Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95. - The original Matplotlib paper. doi.org ↗
- [4] VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media. [Free online](#) ↗
- [5] McKinney, W. (2022). *Python for Data Analysis*, 3rd ed. O'Reilly Media. [Free online](#) ↗

Ngao Labs LMS · Foundations of Data Science & AI · **Week 2 of 10**

Questions? Post in the Week 2 Discussion Forum or reach your mentor during the weekly check-in.